`match-v5.json` reference

`raw-data/match-v5.json` is one League of Legends Match-V5 response. It describes the final state of a single match rather than a timeline of in-game events. The companion `raw-data/match-v5-timeline.json` is the place to look for event-by-event and frame-by-frame data.

This fixture is a completed 5v5 ranked solo game on Summoner's Rift:

Property	Value
Match ID	`NA1_5617625080`
Platform	`NA1`
Queue	`420` (ranked solo/duo)
Map	`11` (Summoner's Rift)
Mode / type	`CLASSIC` / `MATCHED_GAME`
Game version	`16.15.802.4387`
Duration	`1357` seconds (22:37)
Participants / teams	10 / 2

Shape at a glance

```
root
├── metadata
│   ├── dataVersion
│   ├── matchId
│   └── participants[]                # 10 player PUUIDs
└── info
    ├── match metadata and timestamps
    ├── participants[]                # 10 final player-stat records
    │   ├── challenges                # detailed achievement/derived
metrics
    │   ├── missions                  # player-score fields
    │   └── perks                     # rune selections and stat shards
    └── teams[]                       # one record per team (100 and 200)
        ├── bans[]
        └── objectives
```

Array notation in this document means an index: for example, `info.participants[0].championName` is the first participant's champion. Use `[]` when querying every element of that array.

Where to find data

If you need...	Path(s)
The stable match identifier or all player PUUIDs	`metadata.matchId`, `metadata.participants[]`
Match queue, map, mode, version, region/platform, or result	`info.queueId`, `info.mapId`, `info.gameMode`, `info.gameVersion`, `info.platformId`, `info.endOfGameResult`
Match start/end time or duration	`info.gameCreation`, `info.gameStartTimestamp`, `info.gameEndTimestamp`, `info.gameDuration`
A player's account/display identity	`info.participants[].puuid`, `.summonerId`, `.riotIdGameName`, `.riotIdTagline`, `.summonerName`
Champion, level, lane, role, or team	`info.participants[].championId`, `.championName`, `.champLevel`, `.lane`, `.role`, `.teamPosition`, `.individualPosition`, `.teamId`
KDA, win/loss, multikills, or killing sprees	`info.participants[].kills`, `.deaths`, `.assists`, `.win`, `.doubleKills` through `.pentaKills`, `.largestKillingSpree`, `.killingSprees`
Gold, XP, items, summoner spells, or rune setup	`info.participants[].goldEarned`, `.goldSpent`, `.champExperience`, `.item0`–`.item6`, `.summoner1Id`, `.summoner2Id`, `.perks`
Damage, healing, shielding, crowd control, or damage received	`info.participants[].totalDamageDealtToChampions`, `.totalDamageDealt`, `.totalDamageTaken`, `.damageSelfMitigated`, `.totalHeal`, `.totalHealsOnTeammates`, `.totalDamageShieldedOnTeammates`, `.timeCCingOthers`, `.totalTimeCCDealt`
CS and jungle farming	`info.participants[].totalMinionsKilled`, `.neutralMinionsKilled`, `.totalAllyJungleMinionsKilled`, `.totalEnemyJungleMinionsKilled`
Vision and warding	`info.participants[].visionScore`, `.wardsPlaced`, `.wardsKilled`, `.detectorWardsPlaced`, `.visionWardsBoughtInGame`, `.sightWardsBoughtInGame`
Pings and ability/summoner-spell casts	`info.participants[].*Pings`, `.spell1Casts`–`.spell4Casts`, `.summoner1Casts`, `.summoner2Casts`
Turret, inhibitor, dragon, Baron, or objective contribution	player fields such as `.turretKills`, `.dragonKills`, `.baronKills`, `.inhibitorKills`; team totals at `info.teams[].objectives`
Team winner, draft bans, and first-objective ownership	`info.teams[].win`, `.bans[]`, `.objectives.*.first`

If you need...	Path(s)
Derived performance/achievement data	`info.participants[].challenges`
Individual match events, positions, or inventory changes over time	Not in this file; use `raw-data/match-v5-timeline.json`

Top-level sections

`metadata`

`metadata` is the compact identification section:

- `dataVersion` identifies the response data format version (`"2"` here).
- `matchId` is the canonical regional match ID, here `NA1_5617625080`.
- `participants` is an array of the 10 participant PUUIDs. These correspond to the player records in `info.participants`, whose `puuid` field can be used to join the two lists.

`info`

`info` contains the match summary, plus the participant and team arrays.

Its scalar fields are:

- **Lifecycle:** `endOfGameResult`, `gameCreation`, `gameStartTimestamp`, `gameEndTimestamp`, and `gameDuration`. The three time fields are Unix timestamps in milliseconds; `gameDuration` is seconds in this response.
- **Game identity:** `gameId`, `gameName`, `gameMode`, `gameType`, `gameVersion`, `mapId`, `platformId`, `queueId`, and `tournamentCode`.
- **Collections:** `participants` and `teams`.

Participant records: `info.participants[]`

Each of the 10 participant objects has 155 fields in this fixture. They are final post-game statistics, grouped below by purpose. Fields that are zero or empty still appear, so consumers should not treat their presence as proof that the activity occurred.

Identity, assignment, and outcome

- `participantId`, `teamId`, `win`, `placement`, `playerSubteamId`, and `subteamPlacement` identify the player within the game and their result.
- `puuid`, `summonerId`, `summonerName`, `riotIdGameName`, `riotIdTagline`, `summonerLevel`, and `profileIcon` identify the account and current profile presentation. `summonerName` is empty in this sample; use the Riot ID fields for the visible name.
- `championId`, `championName`, `championTransform`, `champLevel`, and `champExperience` describe the champion and its final progression.

- `lane`, `role`, `teamPosition`, `individualPosition`, `positionAssignedByMatchmaking`, and `selectedRolePreferences` contain lane and role assignment data. They may differ, and some can be mode-dependent.

Combat, survival, and spells

- Core score: `kills`, `deaths`, `assists`, `killingSprees`, `largestKillingSpree`, `largestMultiKill`, and `doubleKills`, `tripleKills`, `quadraKills`, `pentaKills`, `unrealKills`.
- Damage dealt: `totalDamageDealt`, `totalDamageDealtToChampions`, `damageDealtToBuildings`, `damageDealtToTurrets`, `damageDealtToObjectives`, `damageDealtToEpicMonsters`, and the matching `magicDamage*`, `physicalDamage*`, and `trueDamage*` breakdowns.
- Damage and recovery: `totalDamageTaken`, `magicDamageTaken`, `physicalDamageTaken`, `trueDamageTaken`, `damageSelfMitigated`, `totalHeal`, `totalHealsOnTeammates`, `totalDamageShieldedOnTeammates`, `totalUnitsHealed`, `totalTimeSpentDead`, `timeCCingOthers`, and `totalTimeCCDealt`.
- Spell usage: `spell1Casts` through `spell4Casts`, `summoner1Id`, `summoner2Id`, `summoner1Casts`, and `summoner2Casts`.

Economy, inventory, and farming

- Economy: `goldEarned`, `goldSpent`, `consumablesPurchased`, and `itemsPurchased`.
- Final inventory: `item0` through `item5` are the six item slots and `item6` is the trinket slot. Values are item IDs, not display names.
- Farming: `totalMinionsKilled`, `neutralMinionsKilled`, `totalAllyJungleMinionsKilled`, and `totalEnemyJungleMinionsKilled`.

Vision, map pressure, objectives, and communication

- Vision: `visionScore`, `wardsPlaced`, `wardsKilled`, `detectorWardsPlaced`, `visionWardsBoughtInGame`, and `sightWardsBoughtInGame`.
- Structures/objectives: `turretKills`, `turretTakedowns`, `turretsLost`, `inhibitorKills`, `inhibitorTakedowns`, `inhibitorsLost`, `nexusKills`, `nexusTakedowns`, `nexusLost`, `dragonKills`, `baronKills`, `objectivesStolen`, and `objectivesStolenAssists`.
- Pings: `allInPings`, `assistMePings`, `basicPings`, `commandPings`, `dangerPings`, `enemyMissingPings`, `enemyVisionPings`, `getBackPings`, `holdPings`, `needVisionPings`, `onMyWayPings`, `pushPings`, `retreatPings`, and `visionClearedPings`.
- Match-end and behavior flags: `gameEndedInEarlySurrender`, `gameEndedInSurrender`, `gameEndedInIGNBSurrender`, `causedGameEndFromIGNBSurrender`, `teamEarlySurrendered`, `teamIGNBSurrendered`, `eligibleForProgression`, `PlayerBehavior`, `wasSevereTransgressor`, `wasPremadeWithSevereTransgressor`, and `wasPremadeWithIGNBGameEndCauser`.

`PlayerScore0`–`PlayerScore11` and `missions.playerScore0`–`playerScore11` are additional score/mission counters. `playerAugment1`–`playerAugment6` and `roleBoundItem` are present for compatibility with game modes that use them; they are not necessarily meaningful for this classic match.

Runes: `info.participants[].perks`

```
perks
├── statPerks
│   ├── defense                    # stat-shard ID
│   ├── flex                       # stat-shard ID
│   └── offense                    # stat-shard ID
└── styles[]
    ├── description                # primaryStyle or subStyle
    ├── style                      # rune-tree ID
    └── selections[]
        ├── perk                   # rune ID
        └── var1, var2, var3       # rune-specific post-game values
```

The response stores numeric IDs; resolve them against the appropriate Riot Data Dragon/static-data version when a display name or icon is needed.

Challenges: `info.participants[].challenges`

`challenges` is a flat object of detailed, mostly derived metrics and achievement flags. This fixture includes 120 fields. It covers:

- **Rates and team share:** `damagePerMinute`, `goldPerMinute`, `kda`, `killParticipation`, `teamDamagePercentage`, `damageTakenOnTeamPercentage`, and vision metrics such as `visionScorePerMinute`.
- **Lane/farming:** `laneMinionsFirst10Minutes`, `jungleCsBefore10Minutes`, `maxCsAdvantageOnLaneOpponent`, `maxLevelLeadLaneOpponent`, `laningPhaseGoldExpAdvantage`, and `visionScoreAdvantageLaneOpponent`.
- **Fights and kills:** `takedowns`, `soloKills`, `outnumberedKills`, `skillshotsHit`, `skillshotsDodged`, `enemyChampionImmobilizations`, `survivedSingleDigitHpCount`, and many timing/context-specific takedown counters.
- **Objectives and vision:** `baronTakedowns`, `dragonTakedowns`, `riftHeraldTakedowns`, `turretTakedowns`, `turretPlatesTaken`, `epicMonsterSteals`, `controlWardsPlaced`, `wardTakedowns`, and related counters.
- **Special modes and achievements:** `SWARM_*`, `HealFromMapSources`, `InfernalScalePickup`, `poroExplosions`, and other cross-mode metrics may be included even when their value is zero for this match.

Treat challenge fields as optional across different matches and game modes, even though all 10 records in this particular file have the same 155 top-level participant fields.

Team records: `info.teams[]`

There are two records, identified by `teamId` (`100` and `200`). Each has:

- `win`: whether that team won.
- `bans[]`: five draft-ban objects in this game. Each object has `championId` and `pickTurn`. These are champion IDs, not names.

- `objectives`: one object per objective type: `atakhan`, `baron`, `champion`, `dragon`, `horde`, `inhibitor`, `riftHerald`, and `tower`. Every objective object contains `kills` (team count) and `first` (whether this team claimed the first one).

For example, the total dragons taken by team 100 are at `info.teams[0].objectives.dragon.kills`; its first-dragon flag is at `info.teams[0].objectives.dragon.first`.

Useful `jq` lookups

Run these from the repository root:

```
# One compact row for every player
jq -r '.info.participants[] | [.teamId, .teamPosition, .riotIdGameName,
  .championName, .kills, .deaths, .assists, .goldEarned, .win] | @tsv' \
  docs/raw-data/match-v5.json

# Team objective totals and first-objective flags
jq '.info.teams[] | {teamId, win, objectives}' docs/raw-data/match-v5.json

# Final item IDs for a player (first participant shown)
jq '.info.participants[0] | [.item0, .item1, .item2, .item3, .item4,
.item5, .item6]' \
  docs/raw-data/match-v5.json

# A participant's selected rune trees and runes
jq '.info.participants[0].perks' docs/raw-data/match-v5.json
```

Important interpretation notes

- IDs such as `championId`, `item0`–`item6`, summoner-spell IDs, and rune IDs require static game data to become human-readable labels.
- Match V5 is a post-game summary. It does not provide the timestamp of an individual kill, purchase, ward placement, or position; consult the timeline file for those questions.
- Some fields exist across multiple game modes and can be zero, empty, or otherwise inapplicable in a standard Summoner's Rift game. Code consuming other matches should tolerate absent optional fields as well.